Q6. Logical-file imaging and integrity

Original question: Create a forensic image of a logical file. Verify the image size, calculate the cryptographic hashes of the original file and its image, compare the results, and explain the importance of evidence integrity.

Aim

Create a raw byte copy of a logical file, compare byte sizes and SHA-256 digests, and explain what integrity verification supports.

Requirements

A stable regular source file, storage space, Bash, dd, stat, sha256sum and cmp. The demonstration below is suitable when the examiner has not supplied a file. For supplied evidence, retain the original and use its documented acquisition workflow.

Procedure

Step 1 - Create a harmless demonstration input, or select the supplied file

```
mkdir -p "$HOME/EHF_Exam/Q6"
cd "$HOME/EHF_Exam/Q6"
printf '%s\n' 'EHF integrity demonstration.' > logical_source.txt
SRC="$PWD/logical_source.txt"
IMG="$PWD/logical_image.dd"
test -f "$SRC" && stat -c '%n : %s bytes' "$SRC"
sha256sum "$SRC" | tee source_before.sha256
```

For a supplied file, omit printf and set SRC to its absolute path. Keep the source stable during acquisition. Ensure IMG is a new, separate output file; dd will overwrite an existing output. Never replace these file paths with device names as a casual extension of this exercise.

Step 2 - Make the byte copy

```
dd if="$SRC" of="$IMG" bs=1M status=progress
```

`if` means input file; `of` means output file; `bs=1M` sets the transfer block size. Check dd's final byte count and return status. Do not use conv=noerror,sync here: padding after read errors could change the intended byte stream. Stop and document a read/write error rather than presenting a successful acquisition.

Step 3 - Verify size, hashes and contents

```
stat -c '%n : %s bytes' "$SRC" "$IMG" | tee sizes.txt
sha256sum "$SRC" "$IMG" | tee hashes_after.txt
sha256sum -c source_before.sha256
cmp "$SRC" "$IMG"
printf 'Comparison exit status: %s\n' "$?"
```

`stat %s` shows logical size in bytes; allocated disk usage can differ. `sha256sum -c` checks that the source still matches its before-copy digest. Compare the two digests in hashes_after.txt; filenames differ but the 64 hexadecimal digest characters should match. `cmp` adds a direct byte comparison.

Expected output - illustrative

```
logical_source.txt : 29 bytes
logical_image.dd : 29 bytes
/path/to/logical_source.txt: OK
Comparison exit status: 0
```

The sample size is for the exact demonstration text plus newline. Actual hash strings are intentionally not invented; record the values generated in your terminal.

Interpretation, observation and result

Matching sizes alone are insufficient. Matching SHA-256 digests strongly support identical file content at the times hashed; a matching cmp confirms no byte difference during its read. Before/after source verification helps detect changes during the workflow, but cannot guarantee a live file never changed and reverted. Record times, source and image paths, both hashes and any errors.

This is a logical-file byte copy, not a full physical disk acquisition. It does not capture the original disk's unallocated space, other files, partition tables, slack outside the logical file, or necessarily filesystem metadata, alternate streams and extended attributes. A raw image of this text file is not a filesystem image to feed to mmls. Matching hashes do not prove provenance, completeness of the selected evidence, absence of earlier tampering or legal admissibility.

Common errors

- Hash mismatch: stop, check source stability, wrong path, truncation and acquisition errors; preserve the failed output and document any repeat.
- Image size differs: check dd completion and free space with df -h.
- Permission denied on supplied evidence: use the lab's read-access procedure; do not change original evidence permissions casually.

Viva questions

- What does SHA-256 return? A 256-bit digest, normally shown as 64 hexadecimal characters.
- Is a matching size enough? No; different files can have the same size.
- Is this a full disk image? No, only the selected logical file's content was copied.
- Does a hash prove authorship? No; it verifies content against a reference, not who created it.